CSE 503 – Advanced Operating System

Rashid Jillani

rjillani@giki.edu.pk

Chapter 2

Processes and Threads

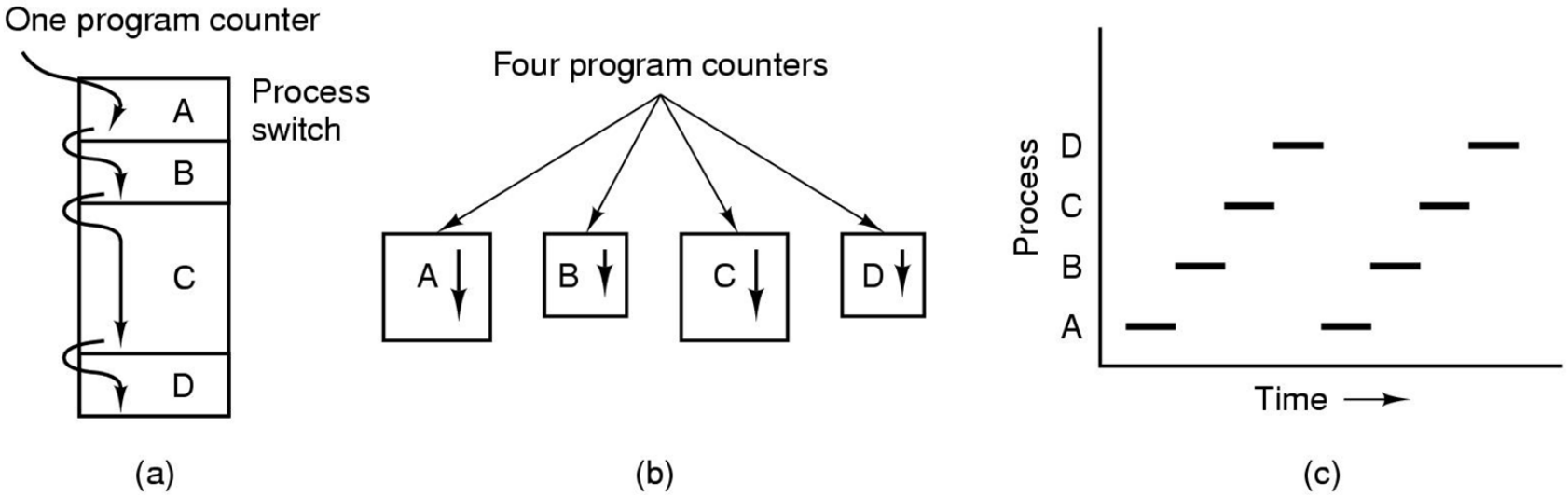
2.1 Processes
2.2 Threads
2.3 Interprocess communication
2.4 Classical IPC problems
2.5 Scheduling

What is a process?

- An instance of a program, replete with registers, variables, and a program counter
- It has a program, input, output and a state
- Why is this idea necessary?
  - A computer manages many computations concurrently- need an abstraction to describe how it does it

Processes

The Process Model



a) Multiprogramming of four programs
b) Conceptual model of 4 independent, sequential processes
c) Only one program active at any instant

Process Creation

Principal events that cause process creation

1. System initialization
2. Execution of a process creation system
3. User request to create a new process
4. Initiation of a batch job

Process Termination

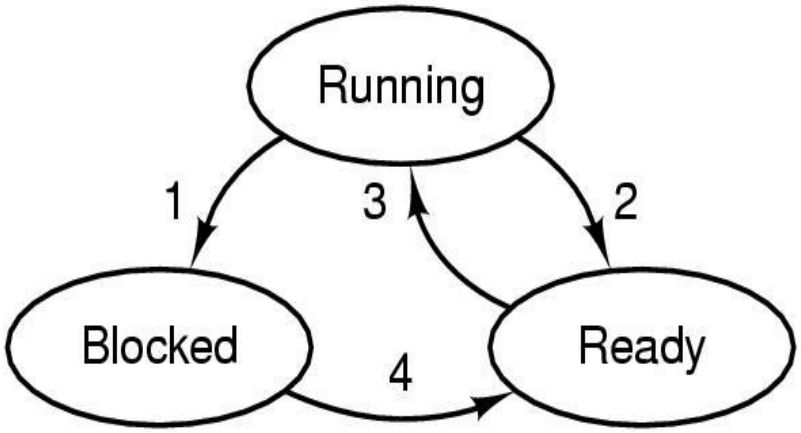
Conditions which terminate processes

1. Normal exit (voluntary)
2. Error exit (voluntary)
3. Fatal error (involuntary)
4. Killed by another process (involuntary)

Process Hierarchies

- Parent creates a child process, child processes can create its own process
- Forms a hierarchy
  - UNIX calls this a "process group“
- Windows has no concept of process hierarchy
  - all processes are created equal

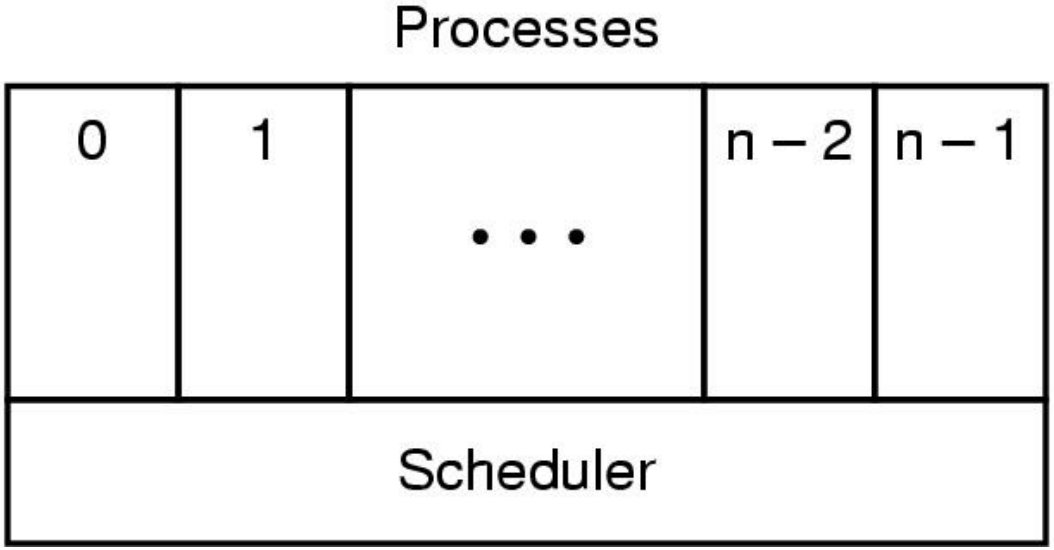
Process States (1)



1. Process blocks for input
2. Scheduler picks another process
3. Scheduler picks this process
4. Input becomes available

- Possible process states
  - running
  - blocked
  - ready
- Transitions between states shown

Process States (2)



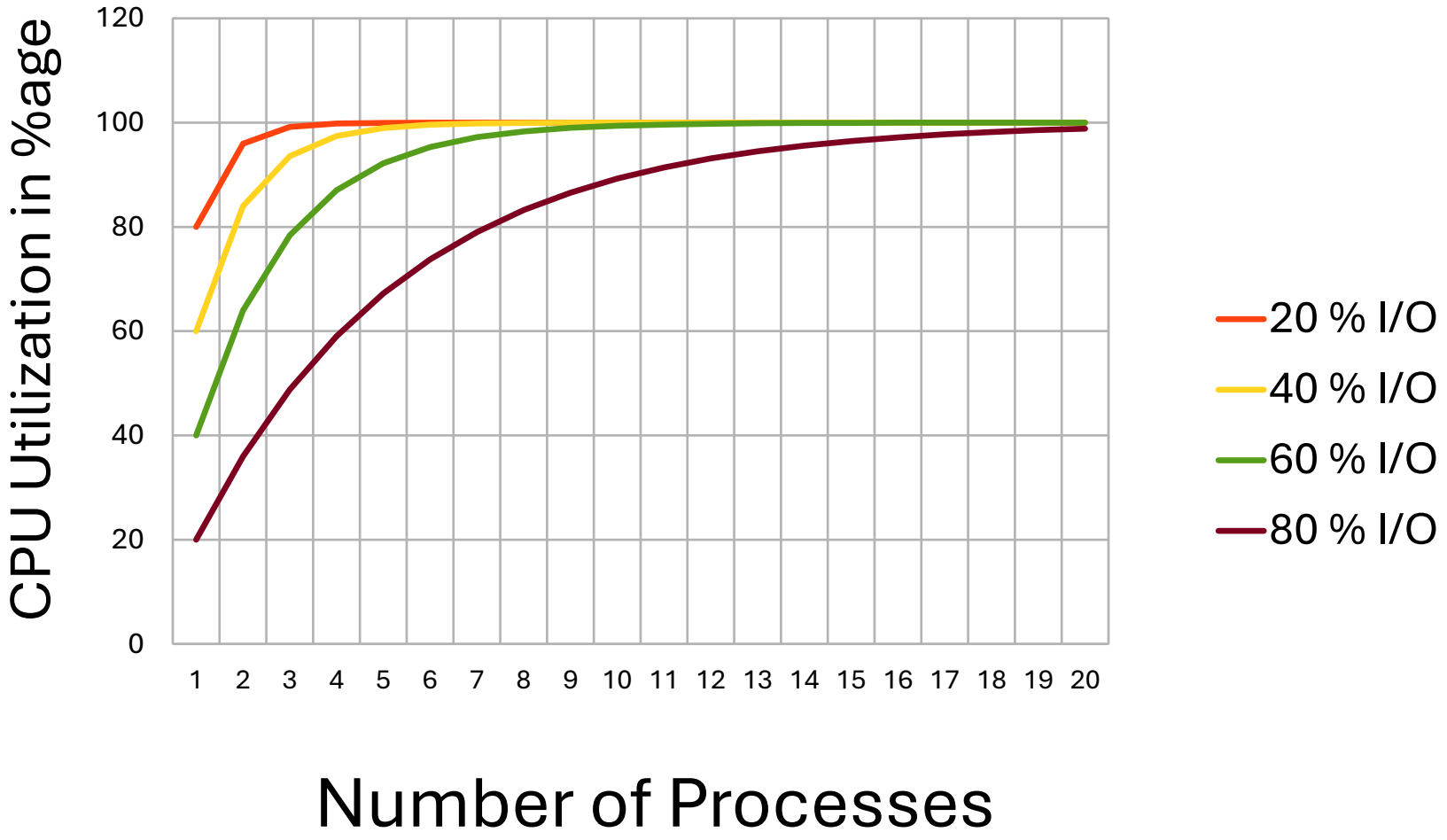
- Lowest layer of process-structured OS
  – handles interrupts, scheduling
- Above that layer are sequential processes

Implementation of Processes (2)

1. Hardware stacks program counter, etc.
2. Hardware loads new program counter from interrupt vector.
3. Assembly language procedure saves registers.
4. Assembly language procedure sets up new stack.
5. C interrupt service runs (typically reads and buffers input).
6. Scheduler decides which process is to run next.
7. C procedure returns to the assembly code.
8. Assembly language procedure starts up new current process.

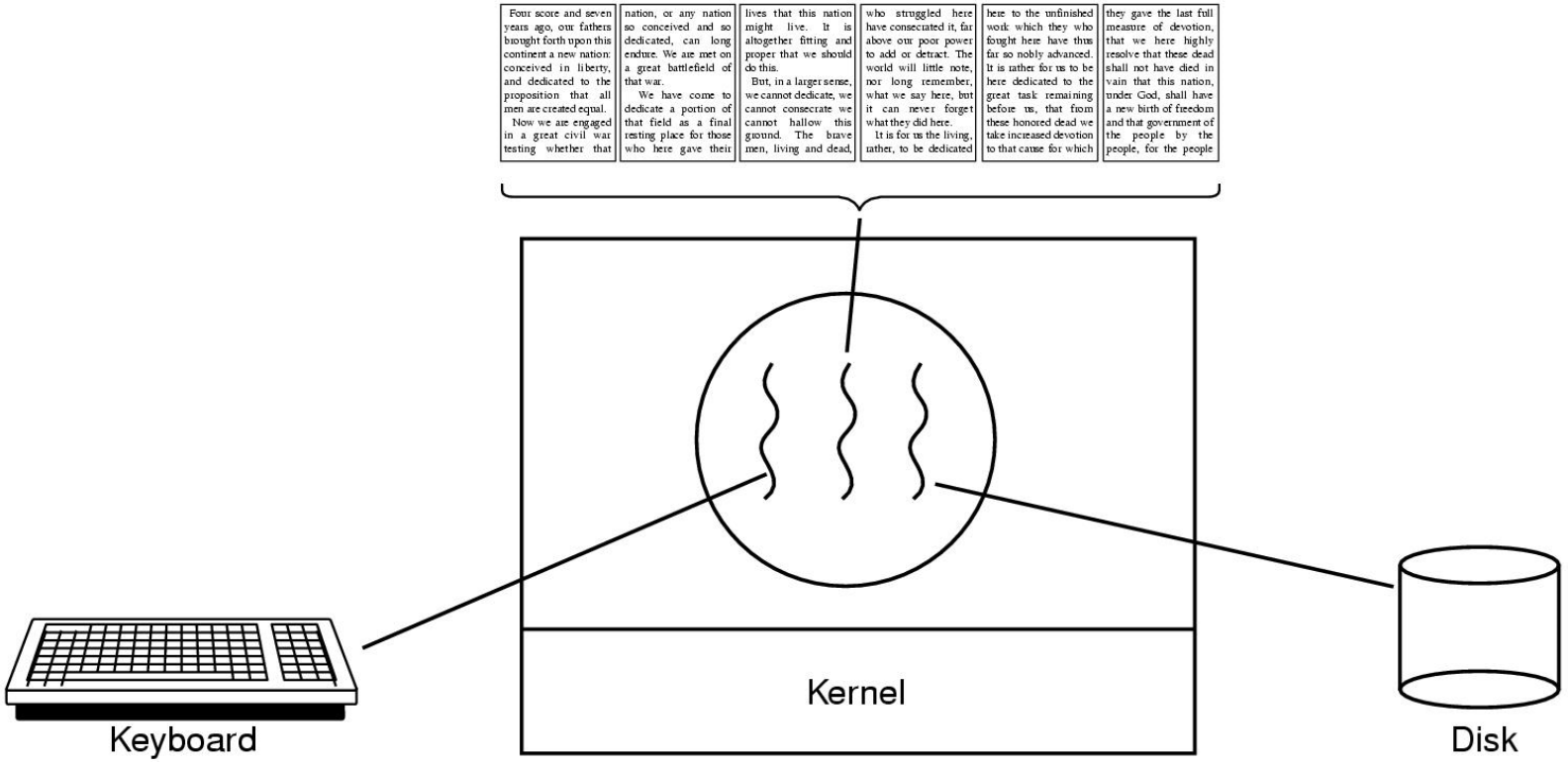
Skeleton of what lowest level of OS does when an interrupt occurs

CPU Utilization



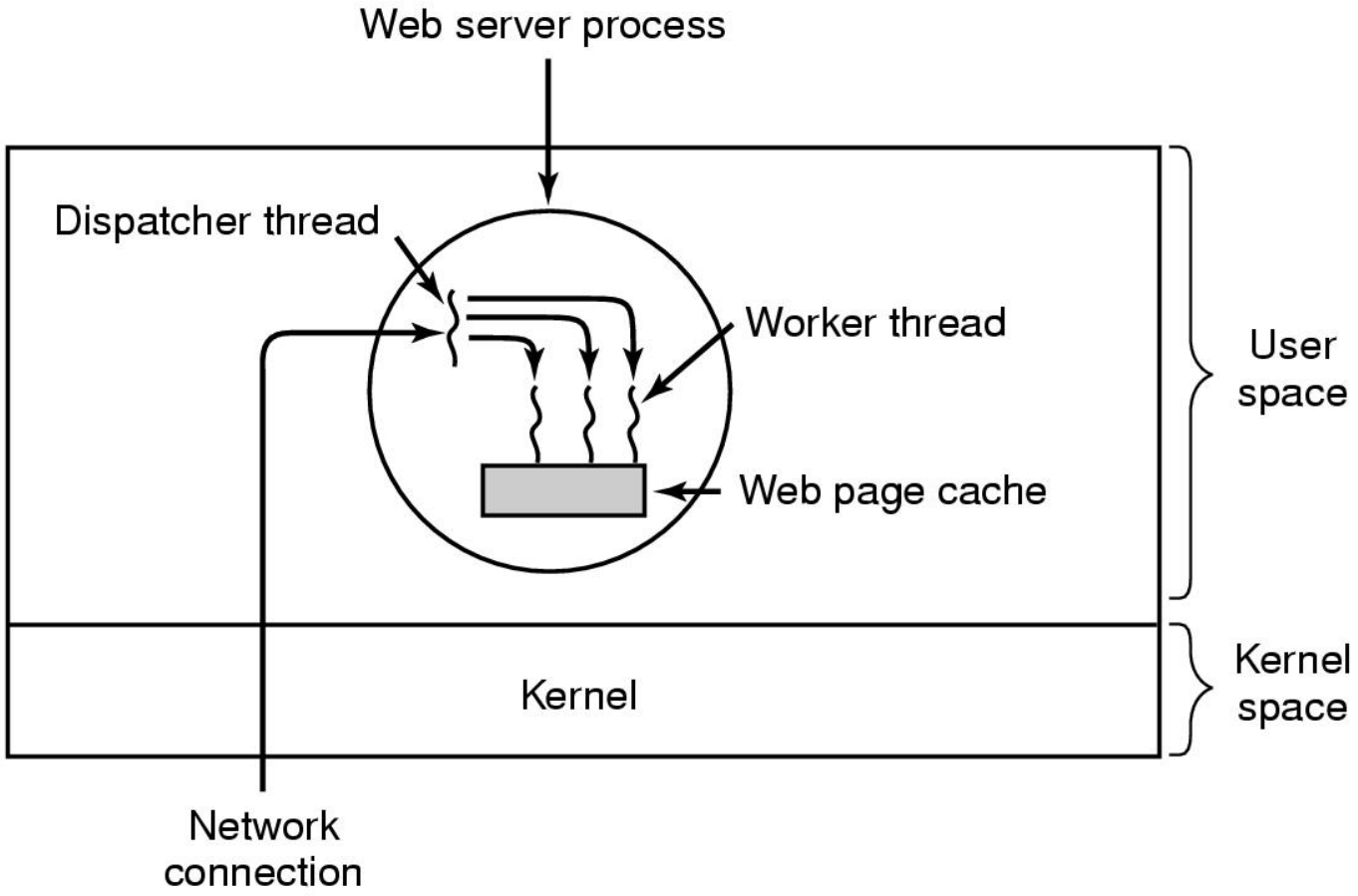
$$CPUUtilization = 1 - P^n$$

Thread Usage (1)



A word processor with three threads

Thread Usage (2)



A multithreaded Web server

Thread Usage (3)

```
while (TRUE) {
  get_next_request(&buf);
  handoff_work(&buf);
}
```

(a)

```
while (TRUE) {
  wait_for_work(&buf)
  look_for_page_in_cache(&buf, &page);
  if (page_not_in_cache(&page)
      read_page_from_disk(&buf, &page);
  return_page(&page);
}
```

(b)

Rough outline of code for previous slide

(a) Dispatcher thread

(b) Worker thread

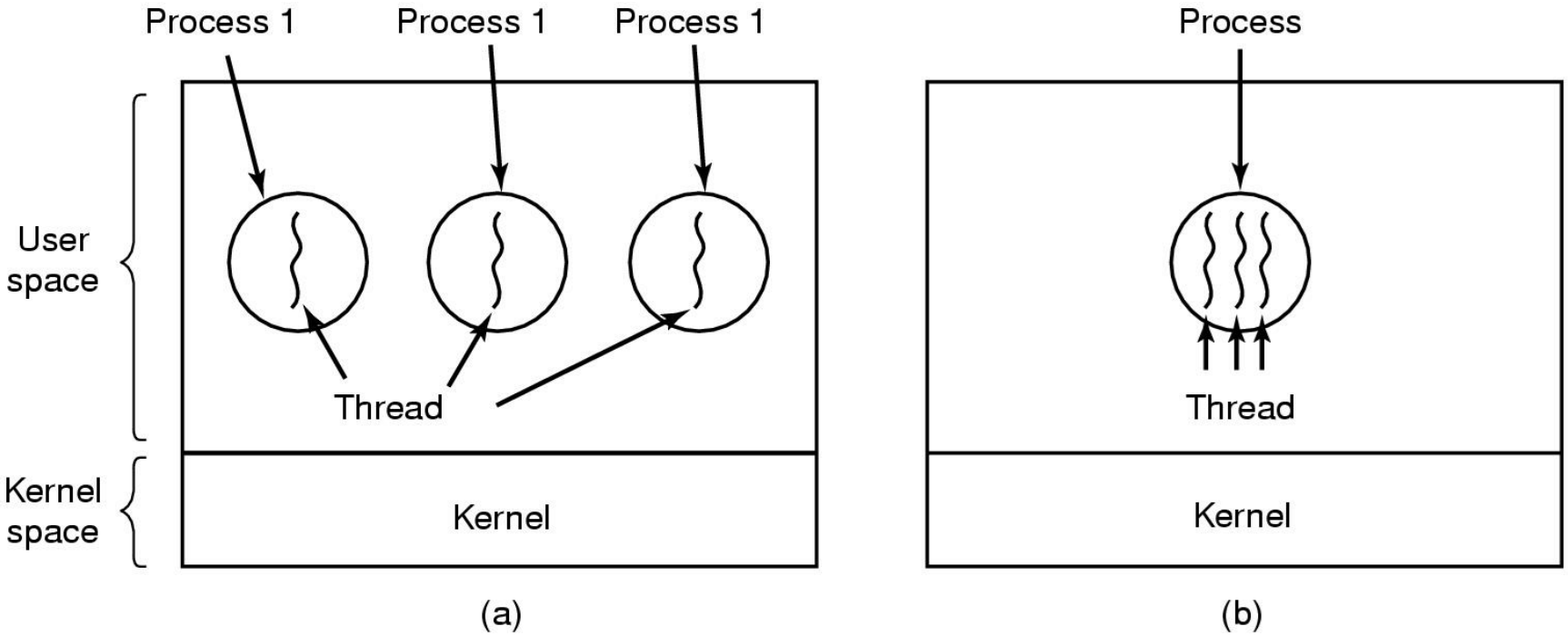
Thread Usage (4)

Model	Characteristics
Threads	Parallelism, blocking system calls
Single-threaded process	No parallelism, blocking system calls
Finite-state machine	Parallelism, nonblocking system calls, interrupts

Three ways to construct a server

Threads

The Thread Model (1)



(a) Three processes each with one thread

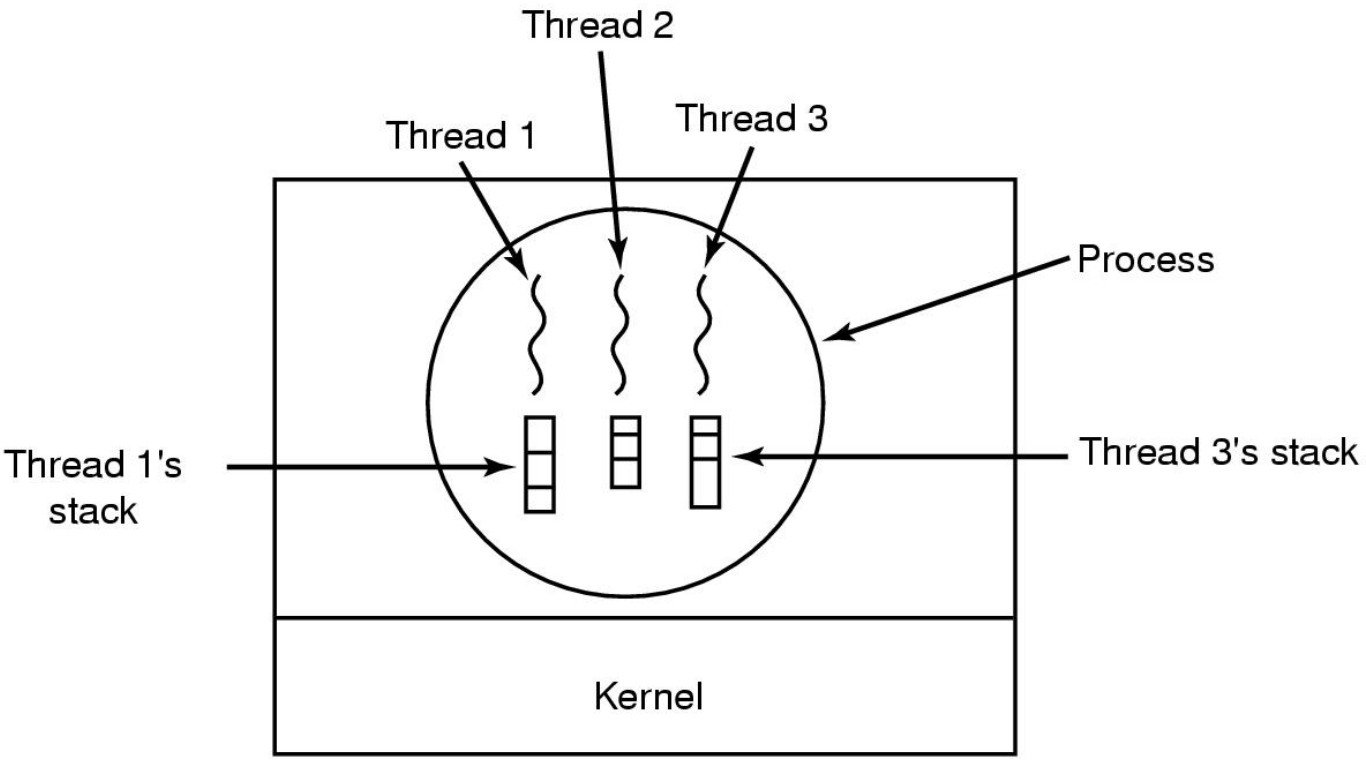
(b) One process with three threads

The Thread Model (2)

Per process items	Per thread items
Address space	Program counter
Global variables	Registers
Open files	Stack
Child processes	State
Pending alarms	
Signals and signal handlers	
Accounting information	

- Items shared by all threads in a process
- Items private to each thread

The Thread Model (3)



Each thread has its own stack

POSIX Thread

Thread call	Description
Pthread_create	Create a new thread
Pthread_exit	Terminate the calling thread
Pthread_join	Wait for a specific thread to exit
Pthread_yield	Release the CPU to let another thread run
Pthread_attr_init	Create and initialize a thread's attribute structure
Pthread_attr_destroy	Remove a thread's attribute structure

Figure 2-14. Some of the Pthreads function calls

An example program

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

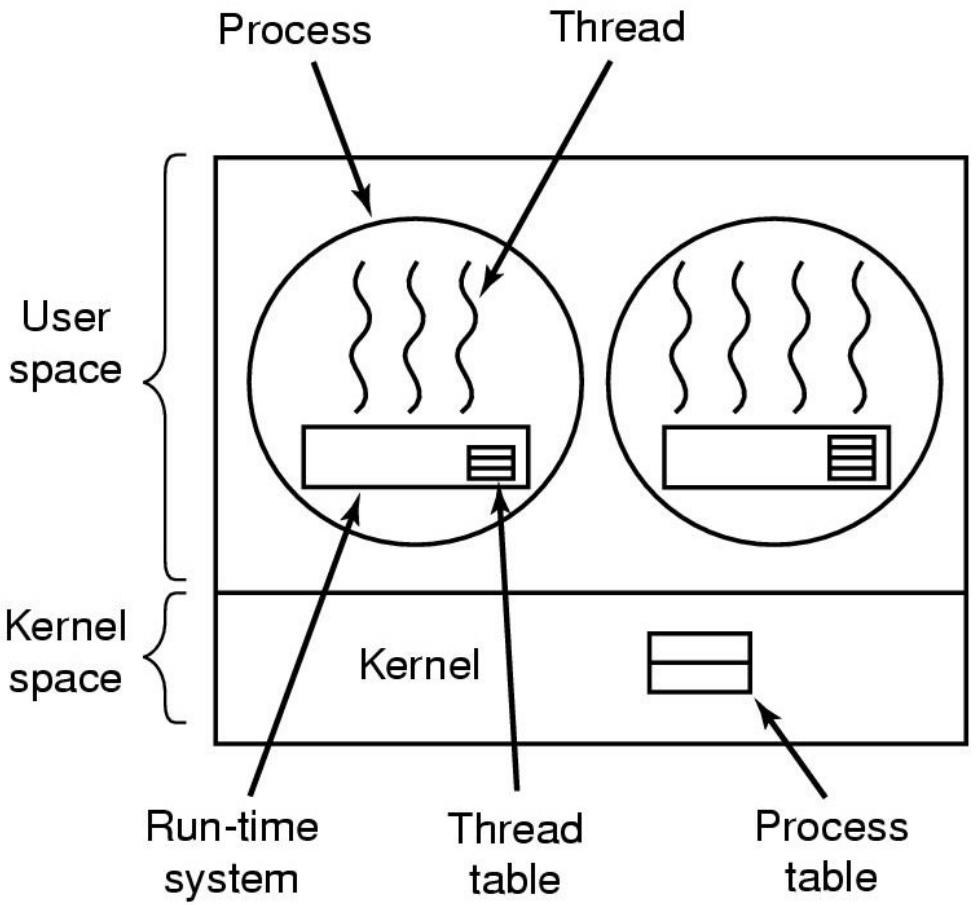
#define NUMBER_OF_THREADS          10

void *print_hello_world (void *tid)
{
        /* This function prints the tread's identifier and then exists. */
        printf("Greetings from thread %d\n, tid);
        pthread_exit(NULL);
}
```

An example program

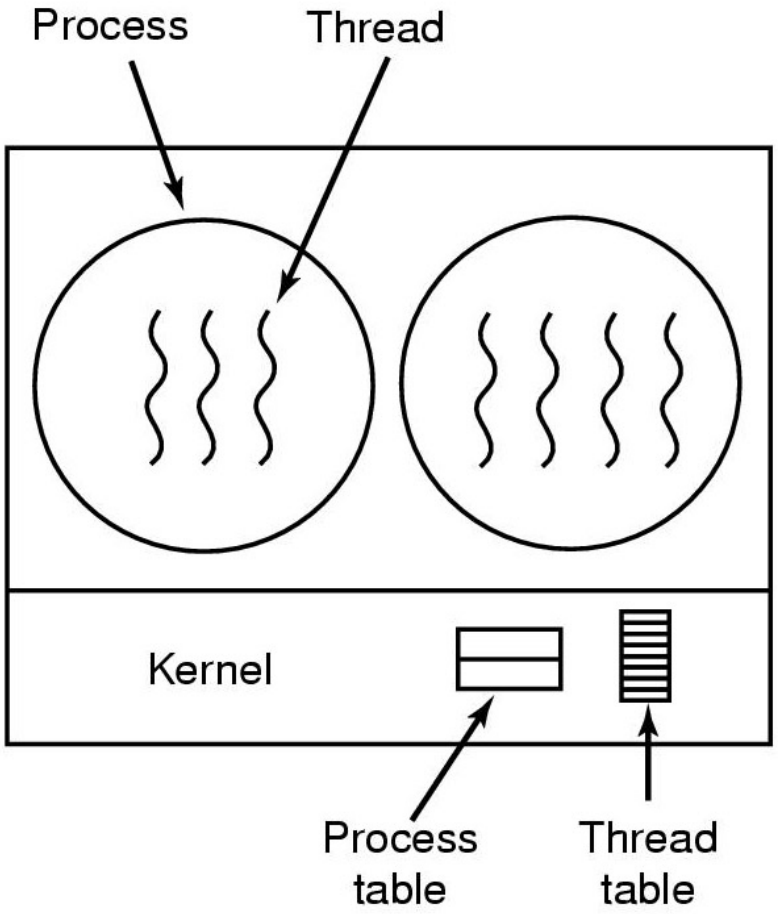
```
int main(int argc, char *argv[]) {
        pthread_t threads [NUMBER_OF_THREADS];
        int status, i;
        for (i = 0; i < NUMBER_OF_THREADS; ++i) {
                printf("Main. Creating thread %d\n, i);
                status = pthread_create(&threads[i], NULL, print_hello_world(void *) i);
                if (status != 0) {
                        printf("oops: pthread_create error code %d\n, status);
                        exit (-1);
                }        // end of if (status != 0)
        } // end of for (i = 0 ....)
        exit (NULL);
}
```

Implementing Threads in User Space



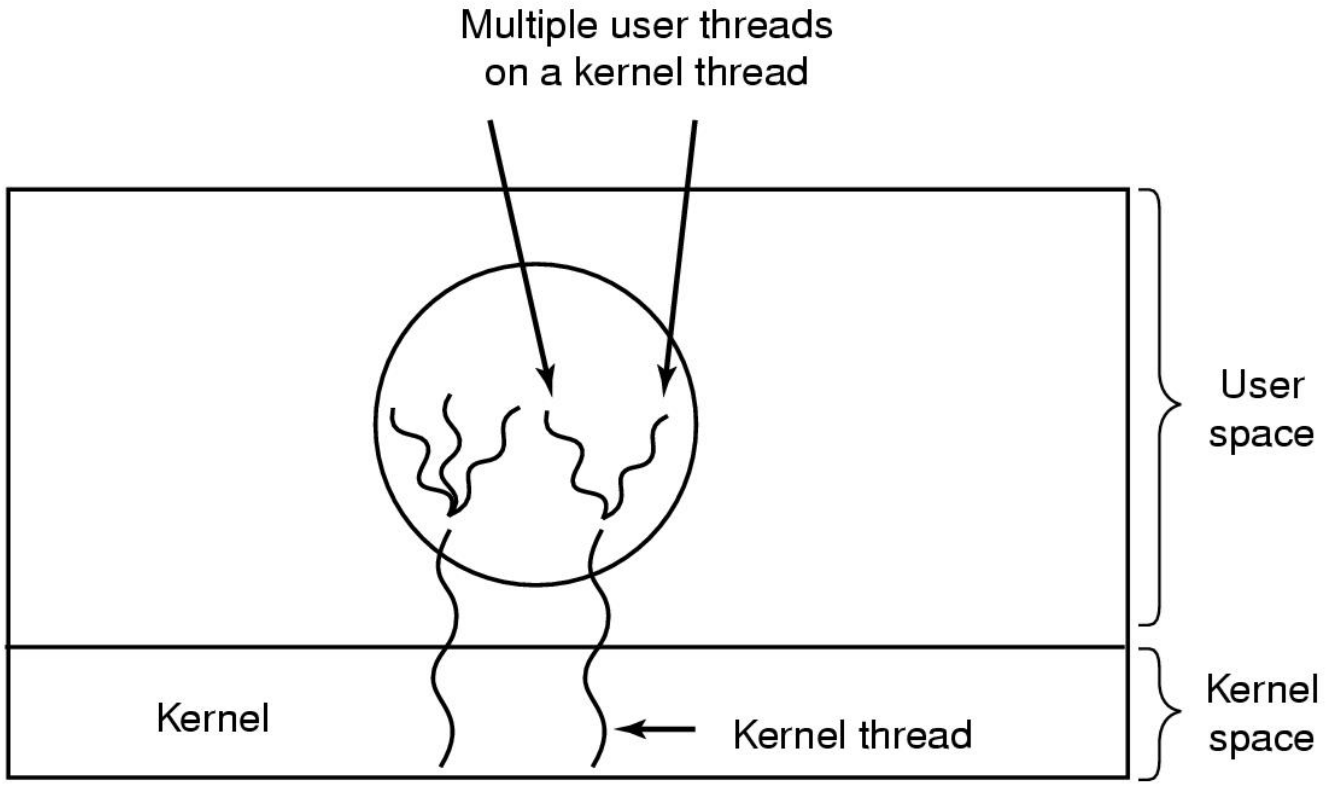
A user-level threads package

Implementing Threads in the Kernel



A threads package managed by the kernel

Hybrid Implementations

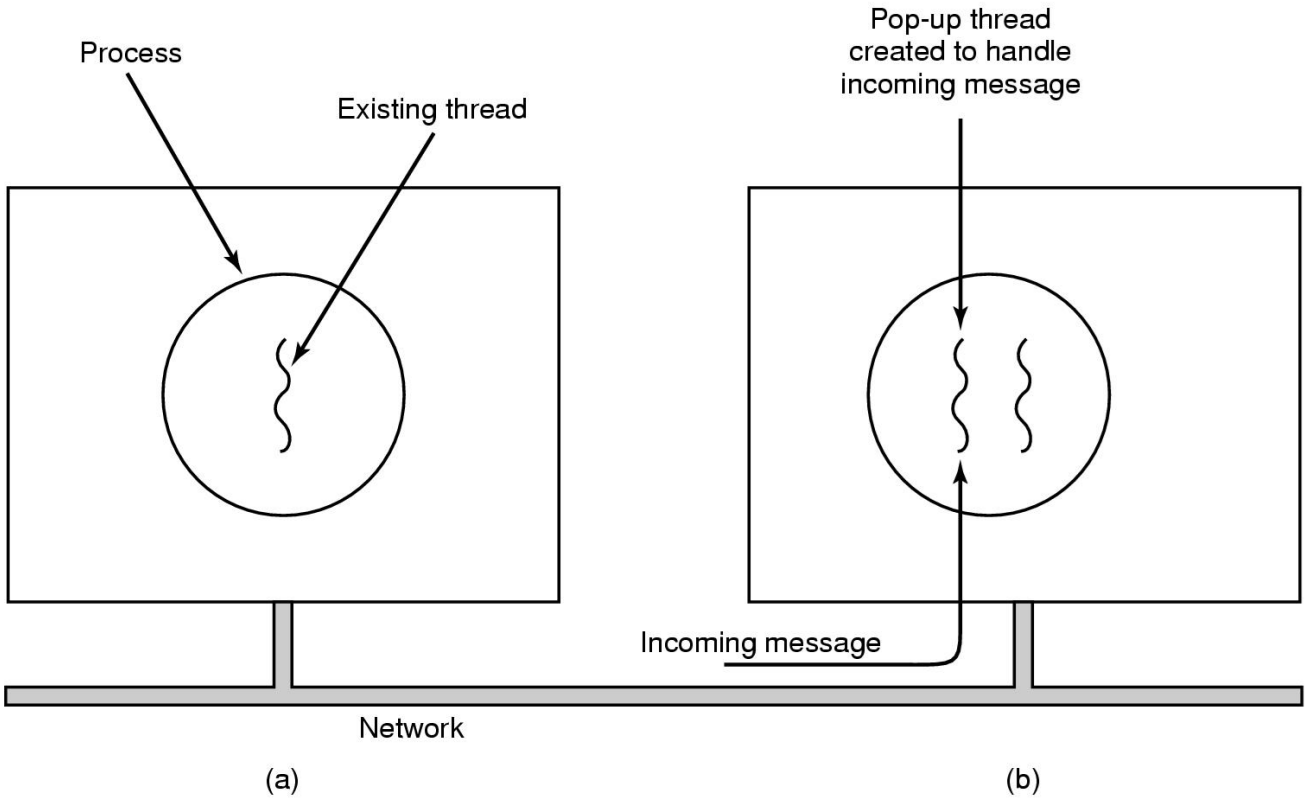


Multiplexing user-level threads onto kernel- level threads

Scheduler Activations

- Goal – mimic functionality of kernel threads
  – gain performance of user space threads
- Avoids unnecessary user/kernel transitions
- Kernel assigns virtual processors to each process
  – lets (user-space) runtime system allocate threads to processors
- Problem:
  Fundamental reliance on kernel (lower layer)
  Calling procedures in user space (higher layer)

Pop-Up Threads

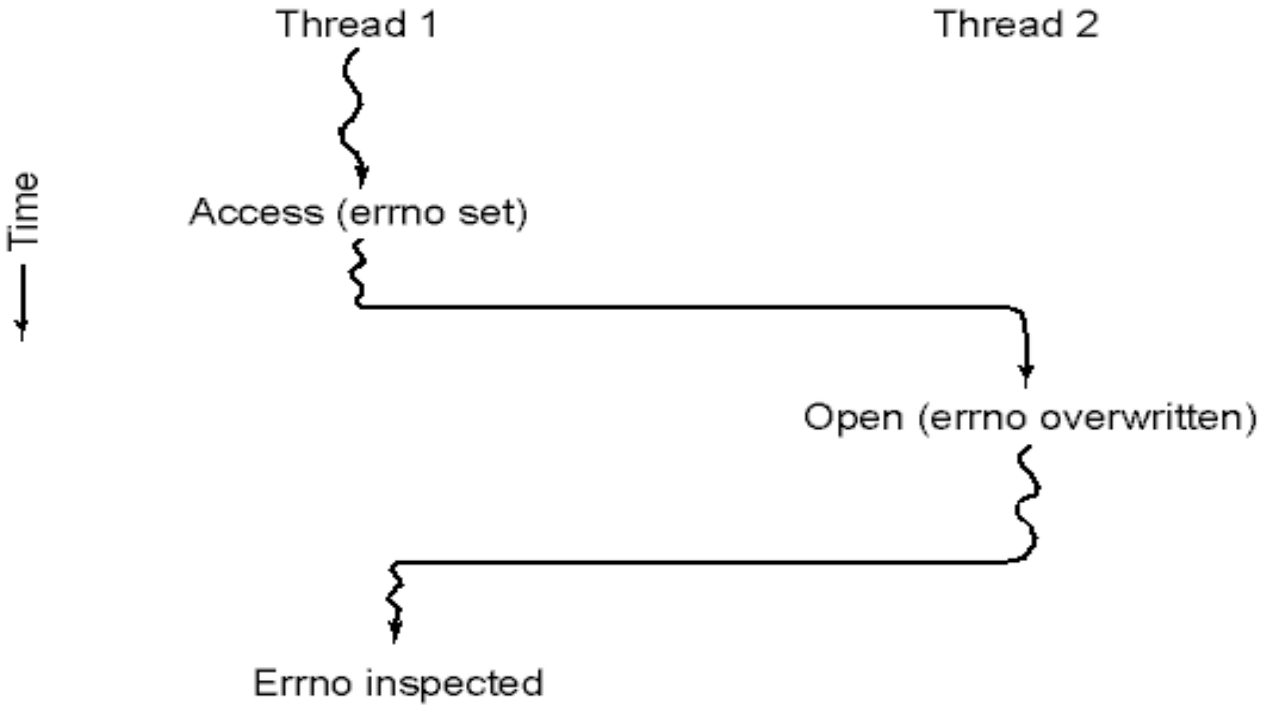


Creation of a new thread when message arrives

(a) before message arrives

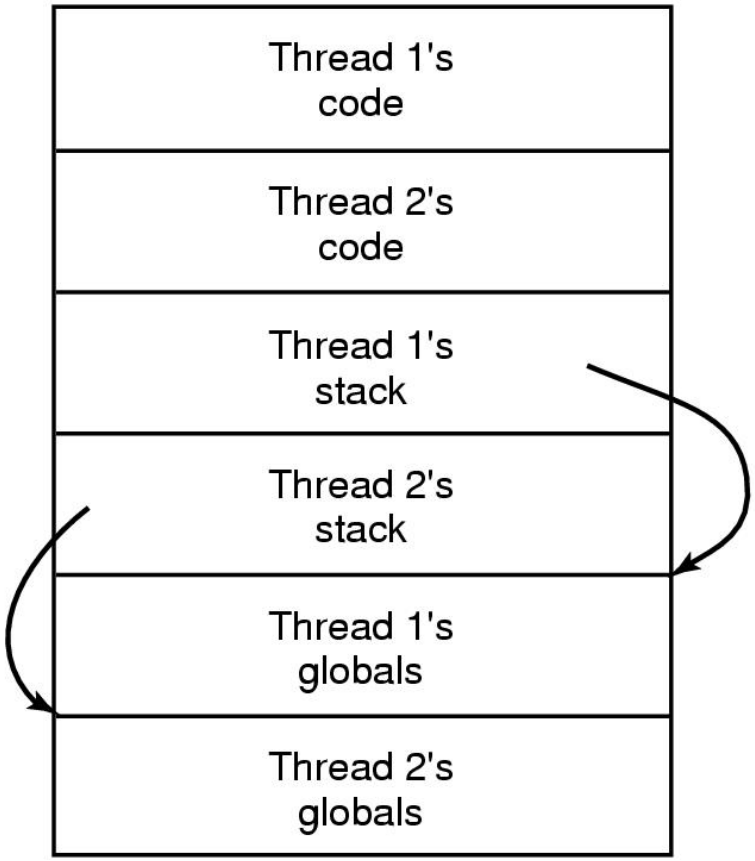
(b) after message arrives

Making Single-Threaded Code Multithreaded (1)



Conflicts between threads over the use of a global variable

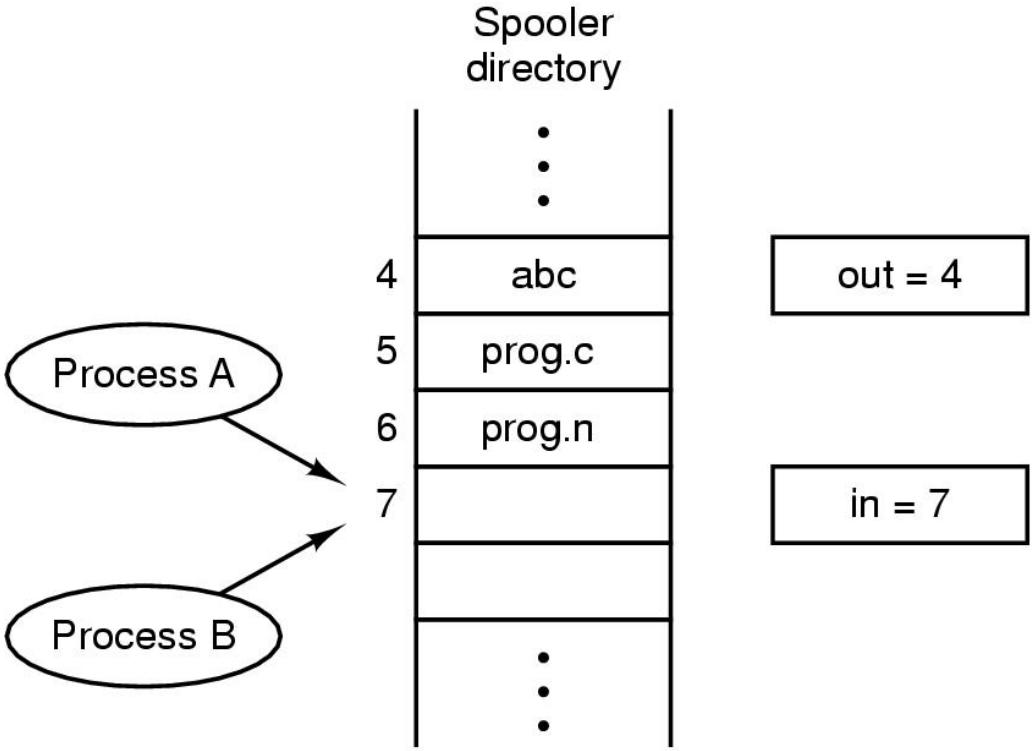
Making Single-Threaded Code Multithreaded (2)



Threads can have private global variables

Interprocess Communication

Race Conditions



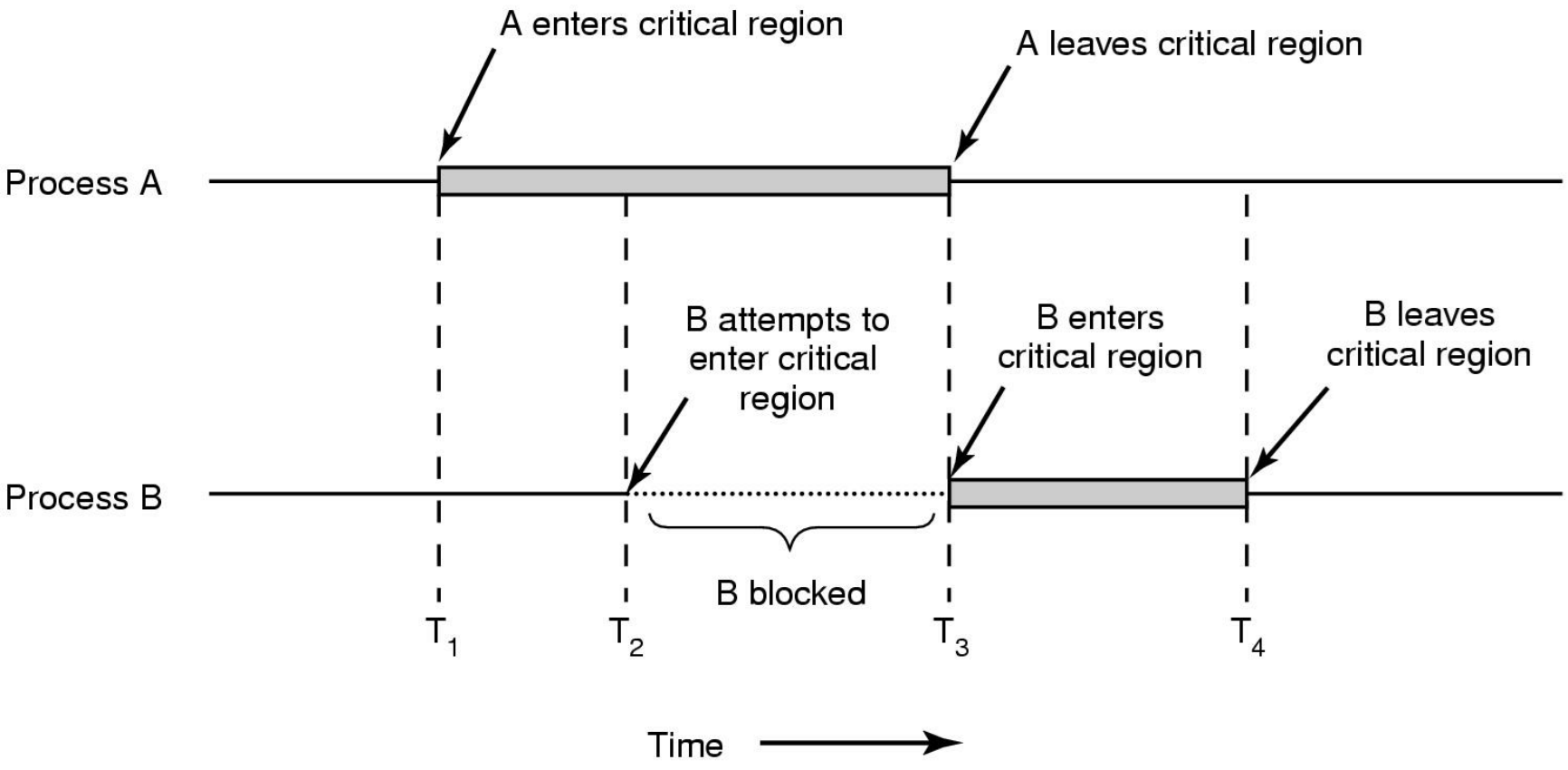
Two processes want to access shared memory at same time

Critical Regions (1)

Four conditions to provide mutual exclusion

1. No two processes simultaneously in critical region
2. No assumptions made about speeds or numbers of CPUs
3. No process running outside its critical region may block another process
4. No process must wait forever to enter its critical region

Critical Regions (2)



Mutual exclusion using critical regions

Mutual Exclusion with Busy Waiting (1)

- Disabling Interrupt
  - User space → Problem
  - Kernel → OK
- Lock Variables
  - Race Condition
- Strict Alternation

Mutual Exclusion with Busy Waiting (2)

```
while (TRUE) {
    while (turn != 0)        /* loop */ ;
    critical_region( );
    turn = 1;
    noncritical_region( );
}
```

(a)

```
while (TRUE) {
    while (turn != 1)        /* loop */ ;
    critical_region( );
    turn = 0;
    noncritical_region( );
}
```

(b)

Proposed solution to critical region problem

(a) Process 0 (b) Process 1

Mutual Exclusion with Busy Waiting (3)

```
#define FALSE  0
#define TRUE   1
#define N      2                        /* number of processes */

int turn;                               /* whose turn is it? */
int interested[N];                      /* all values initially 0 (FALSE) */

void enter_region(int process);         /* process is 0 or 1 */
{
      int other;                        /* number of the other process */

      other = 1 – process;              /* the opposite of process */
      interested[process] = TRUE;       /* show that you are interested */
      turn = process;                   /* set flag */
      while (turn == process && interested[other] == TRUE) /* null statement */ ;
}

void leave_region(int process)          /* process: who is leaving */
{
      interested[process] = FALSE;      /* indicate departure from critical region */
}
```

Peterson's solution for achieving mutual exclusion

Mutual Exclusion with Busy Waiting (4)

```
enter_region:
        TSL REGISTER,LOCK                | copy lock to register and set lock to 1
        CMP REGISTER,#0                  | was lock zero?
        JNE enter_region                 | if it was non zero, lock was set, so loop
        RET | return to caller; critical region entered

leave_region:
        MOVE LOCK,#0                     | store a 0 in lock
        RET | return to caller
```

Entering and leaving a critical region using the TSL instruction

Sleep and Wakeup

The Producer-Consumer Problem

- Bounded-buffer problem
- m producers and n consumers
  - Generalize to 1 producer and 1 consumer
- Producer puts an item into buffer
- Consumers removes an item from buffer
- Problem
  - Buffer full
  - Buffer empty

Sleep and Wakeup

```
#define N 100                                   /* number of slots in the buffer */
int count = 0;                                  /* number of items in the buffer */

void producer(void)
{
     int item;

     while (TRUE) {                             /* repeat forever */
          item = produce_item( );               /* generate next item */
          if (count == N) sleep( );             /* if buffer is full, go to sleep */
          insert_item(item);                    /* put item in buffer */
          count = count + 1;                    /* increment count of items in buffer */
          if (count == 1) wakeup(consumer);     /* was buffer empty? */
     }
}


void consumer(void)
{
     int item;

     while (TRUE) {                             /* repeat forever */
          if (count == 0) sleep( );             /* if buffer is empty, got to sleep */
          item = remove_item( );                /* take item out of buffer */
          count = count - 1;                    /* decrement count of items in buffer */
          if (count == N - 1) wakeup(producer); /* was buffer full? */
          consume_item(item);                   /* print item */
     }
}
```

Producer-consumer problem with fatal race condition

Semaphores

- Dijkstra (1965)
  - Variable (Semaphore) to count the number of wakeups saved for future use
  - Up and Down
  - Or P and V
- Checking and changing value
  - Atomic (Checking the value, changing it, and possibly going to sleep, are all done as a single, indivisible atomic action)
- Proberen (try)
- Verhogen (raise, make higher)

Semaphores

```
#define N 100                                   /* number of slots in the buffer */
typedef int semaphore;                          /* semaphores are a special kind of int */
semaphore mutex = 1;                            /* controls access to critical region */
semaphore empty = N;                            /* counts empty buffer slots */
semaphore full = 0;                             /* counts full buffer slots */

void producer(void)
{
      int item;

      while (TRUE) {                            /* TRUE is the constant 1 */
            item = produce_item( );             /* generate something to put in buffer */
            down(&empty);                       /* decrement empty count */
            down(&mutex);                       /* enter critical region */
            insert_item(item);                  /* put new item in buffer */
            up(&mutex);                         /* leave critical region */
            up(&full);                          /* increment count of full slots */
      }
}


void consumer(void)
{
      int item;

      while (TRUE) {                            /* infinite loop */
            down(&full);                        /* decrement full count */
            down(&mutex);                       /* enter critical region */
            item = remove_item( );              /* take item from buffer */
            up(&mutex);                         /* leave critical region */
            up(&empty);                         /* increment count of empty slots */
            consume_item(item);                 /* do something with the item */
      }
}
```

The producer-consumer problem using semaphores

Mutexes

- Simplified version of Semaphore
  – count is not needed

- Good only for managing mutual exclusion to some shared resource or piece of code

- A mutex is a shared variable that can be in one of two states
  – Locked / Unlocked

Mutexes

```
mutex_lock:
        TSL REGISTER,MUTEX          | copy mutex to register and set mutex to 1
        CMP REGISTER,#0             | was mutex zero?
        JZE ok                      | if it was zero, mutex was unlocked, so return
        CALL thread_yield           | mutex is busy; schedule another thread
        JMP mutex_lock              | try again later
ok:     RET | return to caller; critical region entered

mutex_unlock:
        MOVE MUTEX,#0               | store a 0 in mutex
        RET | return to caller
```

Implementation of *mutex_lock* and *mutex_unlock*

Mutexes in Pthread

Thread Call	Description
Pthread_mutex_init	Create a mutex
Pthread_mutex_destroy	Destroy a mutex
Pthread_mutex_lock	Acquire a lock or block
Pthread_mutex_trylock	Acqure a lock or fail
Pthread_mutex_unlock	Release a lock

Some of the Pthreads calls relating to mutexes

Mutexes in Pthread...

Thread Call	Description
Pthread_cond_init	Create a conditional variable
Pthread_cond_destroy	Destroy a conditional variable
Pthread_cond_wait	Block waiting for signal
Pthread_cond_signal	Signal another thread and wake it up
Pthread_cond_broadcast	Signal multiple threads and wake all of them

Some of the Pthreads calls relating to conditional variables.

Message Passing

```
#define N 100                                   /* number of slots in the buffer */

void producer(void)
{
     int item;
     message m;                                 /* message buffer */

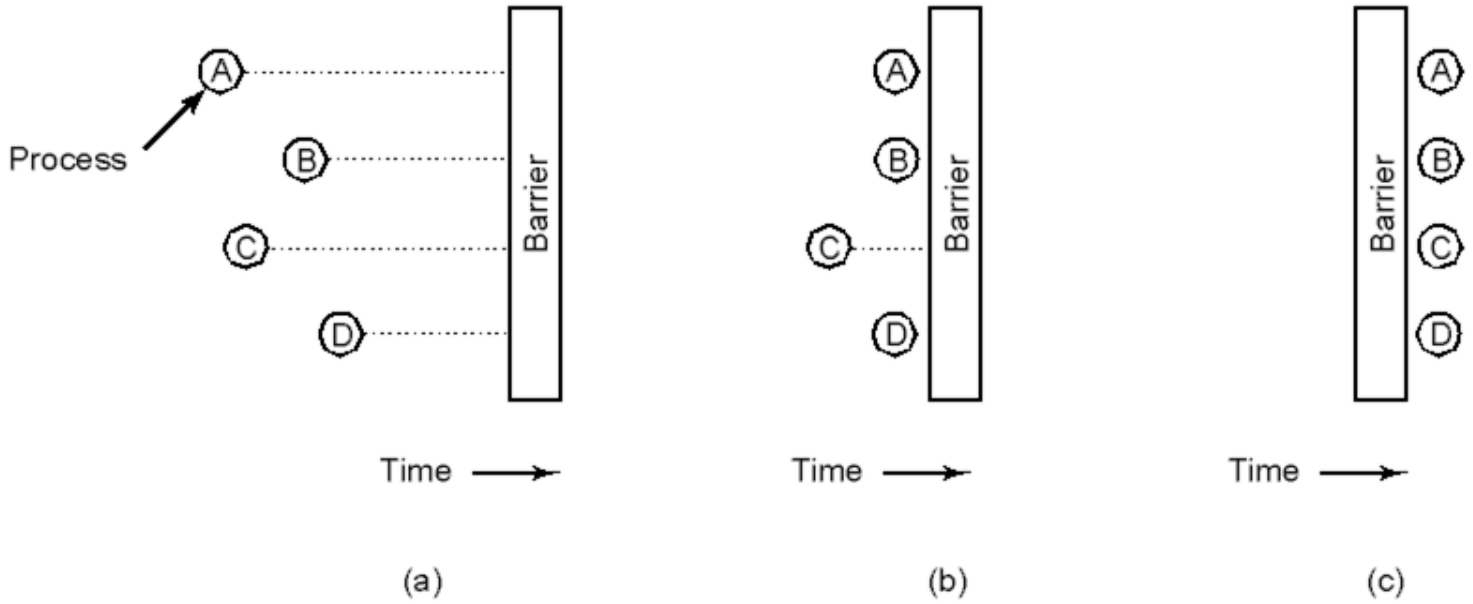
     while (TRUE) {
          item = produce_item( );               /* generate something to put in buffer */
          receive(consumer, &m);                /* wait for an empty to arrive */
          build_message(&m, item);              /* construct a message to send */
          send(consumer, &m);                   /* send item to consumer */
     }
}

void consumer(void)
{
     int item, i;
     message m;

     for (i = 0; i < N; i++) send(producer, &m);  /* send N empties */
     while (TRUE) {
          receive(producer, &m);                /* get message containing item */
          item = extract_item(&m);              /* extract item from message */
          send(producer, &m);                   /* send back empty reply */
          consume_item(item);                   /* do something with the item */
     }
}
```

The producer-consumer problem with N messages

Barriers



Use of a barrier

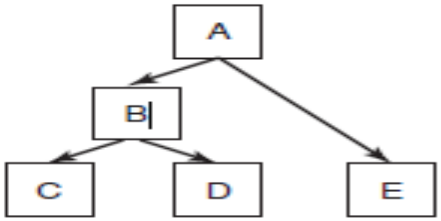
a) processes approaching a barrier
b) all processes but one blocked at barrier
c) last process arrives, all are let through

Avoiding Locks: Read-Copy-Update

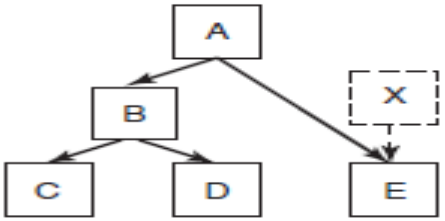
- Fastest lock
  - no lock at all
- Can we allow for concurrent read and write accesses to shared data structures without locking?
  - General case: (No)
  - Some cases: (Yes)

Avoiding Locks: Read-Copy-Update

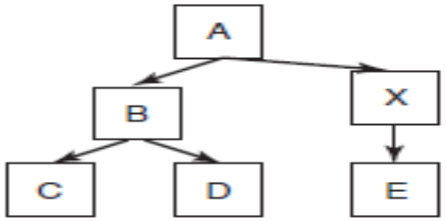
Adding a node:



(a) Original tree.

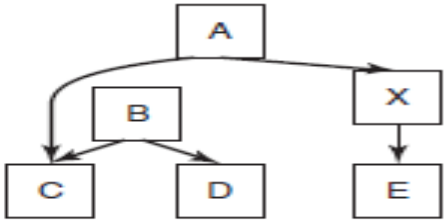


(b) Initialize node X and connect E to X. Any readers in A and E are not affected.

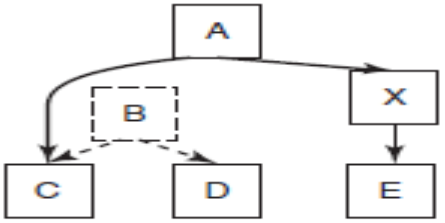


(c) When X is completely initialized, connect X to A. Readers currently in E will have read the old version, while readers in A will pick up the new version of the tree.

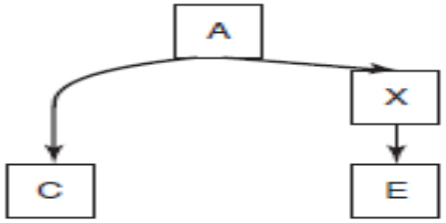
Removing nodes:



(d) Decouple B from A. Note that there may still be readers in B. All readers in B will see the old version of the tree, while all readers currently in A will see the new version.



(e) Wait until we are sure that all readers have left B and C. These nodes cannot be accessed any more.



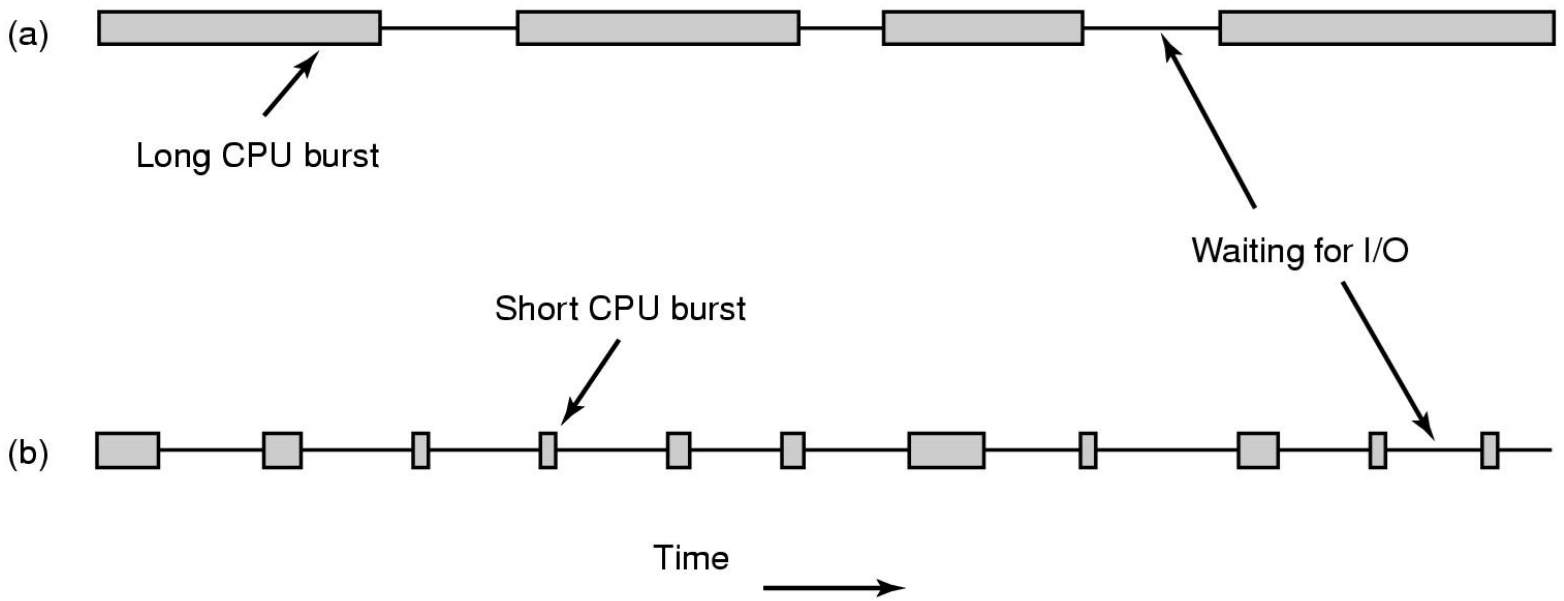
(f) Now we can safely remove B and D

Read-Copy-Update: inserting a node in the tree and then removing a branch—all without locks.

Scheduling

- Multiprogramming → Multiple processes and Threads
- Schedular
- Scheduling Algorithm

Introduction to Scheduling (1)



- Bursts of CPU usage alternate with periods of I/O wait
  a) a CPU-bound process
  b) an I/O bound process

Introduction to Scheduling (2)

- When to schedule?
- Preemptive scheduling
- Non-preemptive scheduling

Introduction to Scheduling (3)

All systems

- Fairness - giving each process a fair share of the CPU
- Policy enforcement - seeing that stated policy is carried out
- Balance - keeping all parts of the system busy

Batch systems

- Throughput - maximize jobs per hour
- Turnaround time - minimize time between submission and termination
- CPU utilization - keep the CPU busy all the time

Interactive systems

- Response time - respond to requests quickly
- Proportionality - meet users' expectations

Real-time systems

- Meeting deadlines - avoid losing data
- Predictability - avoid quality degradation in multimedia systems

Scheduling Algorithm Goals

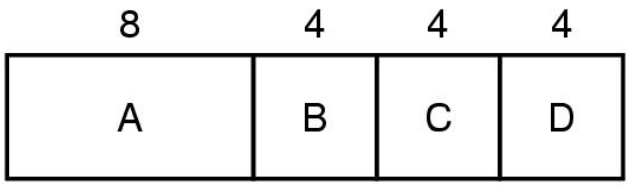
Introduction to Scheduling (4)

- **Throughput**
  - The number of jobs per hour that the system completes.
- **Turnaround time**
  - Statistically average time from the moment that a batch job is submitted until the moment it is completed.
  - Measures how long the average user has to wait for the output. Rule is: Small is Beautiful.
- **Response time**
  - Time between issuing a command and getting the result.
- **Proportionality**
- **CPU Utilization**

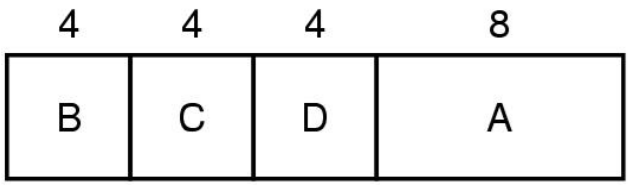
Scheduling in Batch Systems

- First Come First Serve (FCFS)
- Shortest Job First (SJF)
- Shortest Remaining Time Next

Shortest Job First



(a)



(b)

An example of shortest job first scheduling

a) Running four jobs in the original order
b) Running them in SJF order

SJF vs. FCFS

- A = 5 units
- B = 10 units
- C = 5 units
- D = 1 units

FCFS

– Turnaround time = (5 + 15 + 20 + 21) /4 = 15.25

SJF

– Turnaround time = (1+6+11+21)/4 = 9.75

Scheduling in Interactive Systems

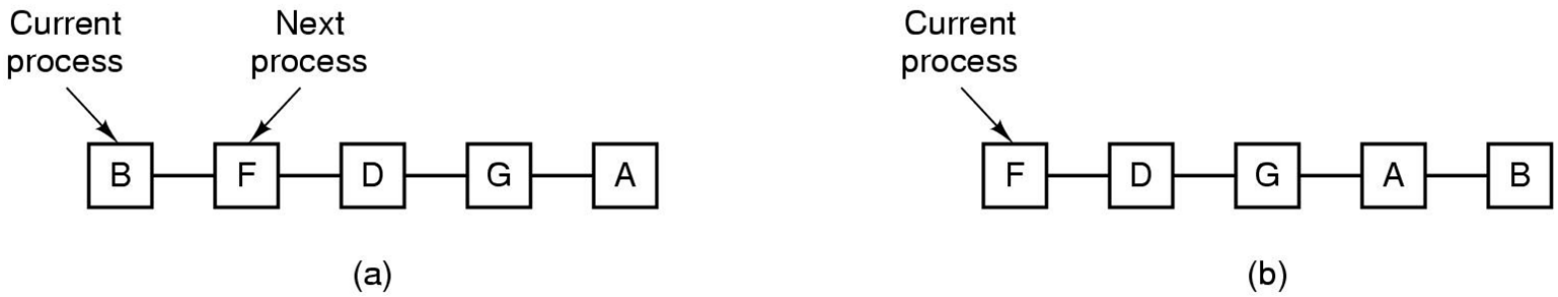
- Round Robin Scheduling
- Priority Scheduling
- Multiple Queues
- Shortest Process Next
- Guaranteed Scheduling
- Lottery Scheduling
- Fair-Share Scheduling

Round Robin Scheduling

- Process switch or context switch
- Quantum
- Process switch / Context switch
  – takes 1 msec
- Quantum
  – 4 msec
- Administration over head is 20%

- What happens if Quantum is 100 msec?

Round Robin Scheduling

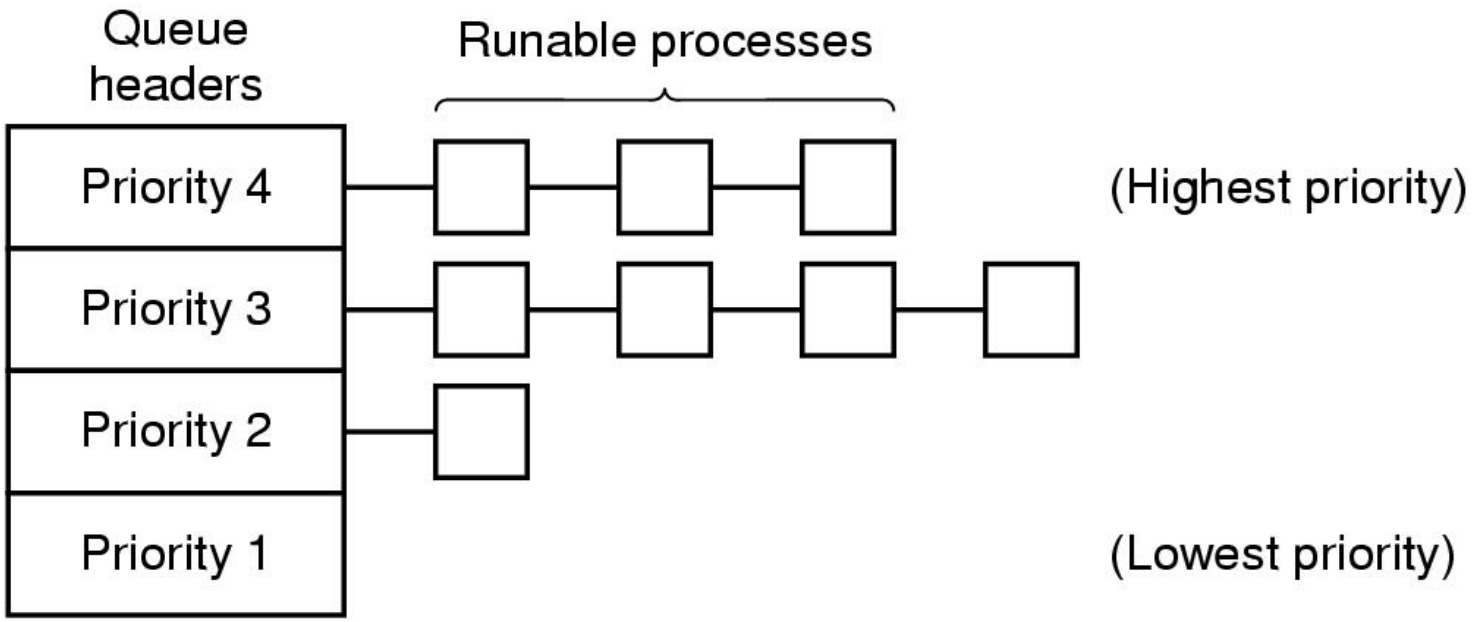


Round Robin Scheduling

a) list of runnable processes

b) list of runnable processes after B uses up its quantum

Priority Scheduling



A scheduling algorithm with four priority classes

Priority = 1/f, where f is the fraction of last quantum

Multiple Queues

- CPU-bound process should get large quanta
- Large quanta means poor response time
- User priority classes
- One, two, four quanta and so on
- Example:- a process of 100 quanta
  - 1, 2, 4, 8, 16, 32, 64
  - Uses 37 of 64 quanta to finish
  - 7 swaps are needed instead of 100
- A process moved down, if it used up all quanta

Shortest Process Next

- Estimated Time
  - $\alpha T_0 + (1 - \alpha) T_1$
- Let $\alpha = ½$
  - T_0 ,
  - $T_0/2 + T_1/2$,
  - $T_0/4 + T_1/4 + T_2/2$,
  - $T_0/8 + T_1/8 + T_2/4$, $T_3/2$
  - $T_0/16 + T_1/16 + T_2/8$, $T_3/4$, $T_4/2$
- Aging

Guaranteed Scheduling

- n users
  - 1/n of the CPU power
- Similarly on a single user system
  - n processes should get 1/n of the CPU cycles
- CPU Time consumed : CPU Time entitled
- A ratio of 2 means twice
- A ratio of 0.5 means half

Lottery Scheduling

- Give lottery ticket against each resource (e.g. CPU) to a process
- A lottery ticket is chosen at random
- System holds a lottery 50 times a second, with each winner getting 20 msec of CPU time as a prize
- “All processes are equal, but some processes are more equal.” by George Orwell
- More important processes can be given extra tickets, to increase their odds of winning.
- Fraction f ticket means f fraction of resource
- 10, 20, and 25 tickets means 10:20:25 proportion of CPU cycle using probabilistic
- It also solve the problem of starvation

Fair-Share Scheduling

- User 1
  - 9 processes, 90% CPU
- User 2
  - 1 Process, 10% CPU
- A B C & D for user 1 and E for user 2
- 50% usage
  - A E B E C E D E A E B E C E D E …
- User 1 is twice to user 2
  - A B E C D E A B E C D E ...

Real-Time Systems

- Voice from CD-ROM
- Hospital intensive-care unit
- Autopilot
- Robot control in in automated factory
- Hard real time
  - There are absolute deadlines that must be met
- Soft real time
  - Missing an occasional deadline is undesirable, but the nevertheless tolerable

Real-Time Systems...

- Periodic Events
  - they occur at regular intervals
- Aperiodic Events
  - they occur unpredictably

Scheduling in Real-Time Systems

Some time it may not be possible to handle all periodic events, but it depends on time of event requires for processing

- Given
  - – *m* periodic events
  - – event *i* occurs with period P_i and requires C_i seconds
- Then the load can only be handled if $\sum_{i=1}^{m} \frac{C_i}{P_i} \leq 1$
- Meeting this criterion is said to be Schedulable

Example

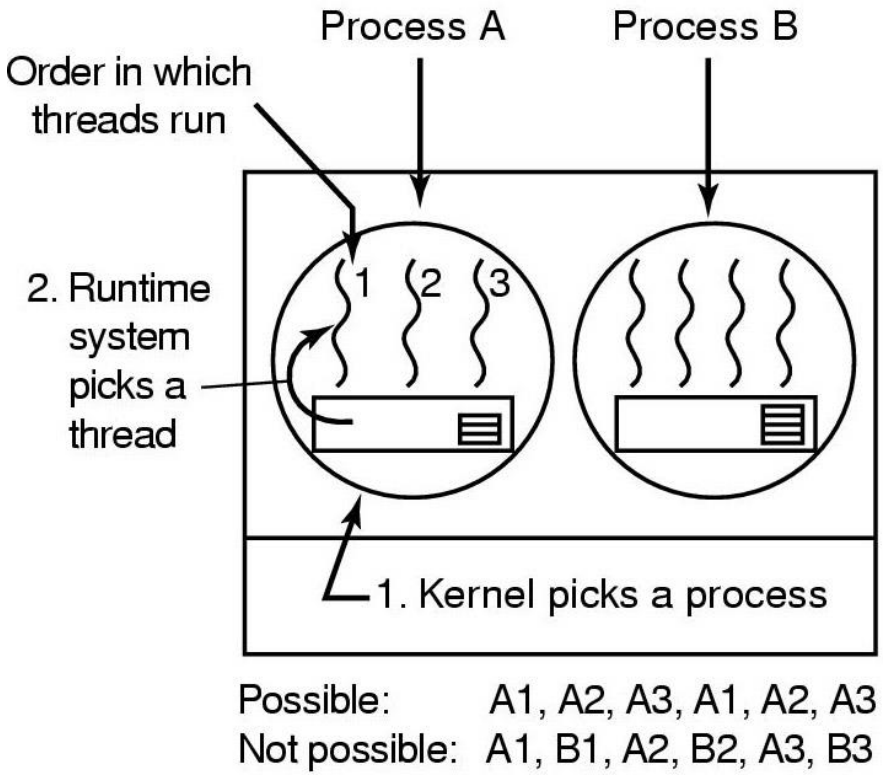
- Consider a soft real-time system with three periodic events
  - 100, 200, and 500 msecs, respectively
- If these events require
  - 50, 30 and 100 msecs of CPU time per event
- The system is schedulable
  - As $0.5 + 0.15 + 0.2 < 1$

- Static/Dynamic Realtime Scheduling Algorithms

Policy Versus Mechanism

- Separate what is allowed to be done with how it is done
  – a process knows which of its children threads are important and need priority
- Scheduling algorithm parameterized
  – mechanism in the kernel
- Parameters filled in by user processes
  – policy set by user process using system call

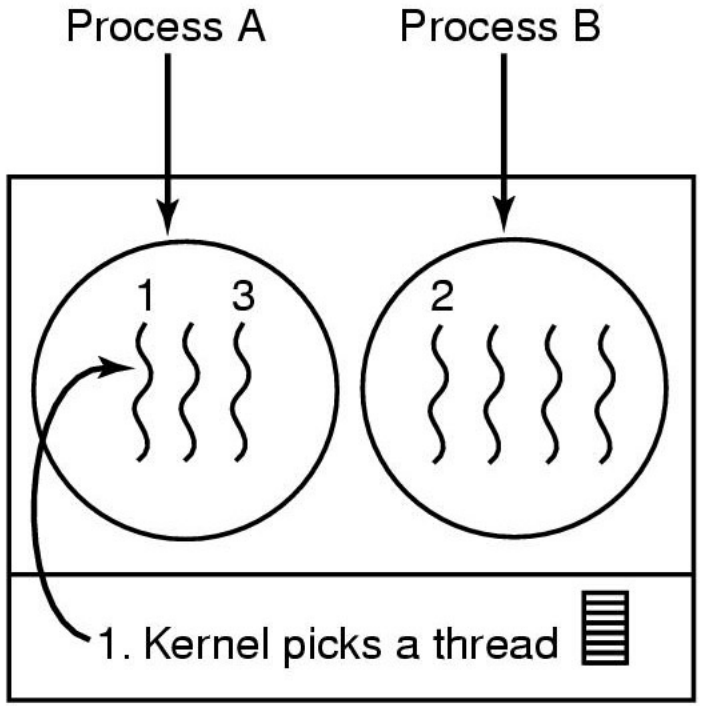
Thread Scheduling (1)



Possible scheduling of user-level threads

- 50-msec process quantum
- threads run 5 msec/CPU burst

Thread Scheduling (2)



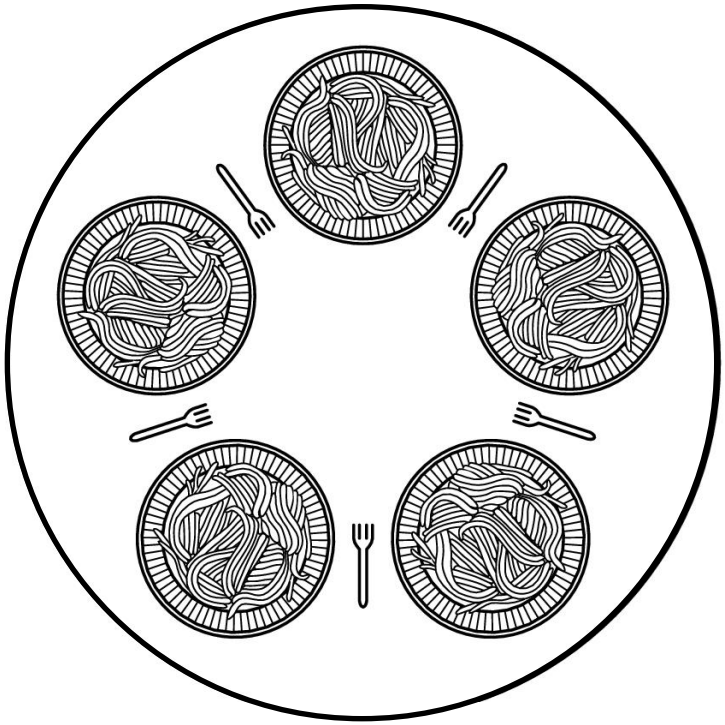
Possible: A1, A2, A3, A1, A2, A3
Also possible: A1, B1, A2, B2, A3, B3

Possible scheduling of kernel-level threads

- 50-msec process quantum
- threads run 5 msec/CPU burst

Dining Philosophers (1)

Synchronization Problem



- Philosophers eat/think
- Eating needs 2 forks
- Pick one fork at a time

- How to prevent deadlock

Dining Philosophers (2)

```
#define N 5                                     /* number of philosophers */

void philosopher(int i)                         /* i: philosopher number, from 0 to 4 */
{
      while (TRUE) {
            think( );                           /* philosopher is thinking */
            take_fork(i);                       /* take left fork */
            take_fork((i+1) % N);               /* take right fork; % is modulo operator */
            eat( );                             /* yum-yum, spaghetti */
            put_fork(i);                        /* put left fork back on the table */
            put_fork((i+1) % N);                /* put right fork back on the table */
      }
}
```

A nonsolution to the dining philosophers problem

Dining Philosophers (3)

```
#define N                5                  /* number of philosophers */
#define LEFT             (i+N–1)%N          /* number of i's left neighbor */
#define RIGHT            (i+1)%N            /* number of i's right neighbor */
#define THINKING         0                  /* philosopher is thinking */
#define HUNGRY           1                  /* philosopher is trying to get forks */
#define EATING           2                  /* philosopher is eating */
typedef int semaphore;                      /* semaphores are a special kind of int */
int state[N];                               /* array to keep track of everyone's state */
semaphore mutex = 1;                        /* mutual exclusion for critical regions */
semaphore s[N];                             /* one semaphore per philosopher */

void philosopher(int i)                     /* i: philosopher number, from 0 to N–1 */
{
      while (TRUE) {                        /* repeat forever */
            think( );                       /* philosopher is thinking */
            take_forks(i);                  /* acquire two forks or block */
            eat( );                         /* yum-yum, spaghetti */
            put_forks(i);                   /* put both forks back on table */
      }
}
```

Solution to dining philosophers problem (part 1)

Dining Philosophers (4)

```
void take_forks(int i)                  /* i: philosopher number, from 0 to N–1 */
{
      down(&mutex);                     /* enter critical region */
      state[i] = HUNGRY;                /* record fact that philosopher i is hungry */
      test(i);                          /* try to acquire 2 forks */
      up(&mutex);                       /* exit critical region */
      down(&s[i]);                      /* block if forks were not acquired */
}

void put_forks(i)                       /* i: philosopher number, from 0 to N–1 */
{
      down(&mutex);                     /* enter critical region */
      state[i] = THINKING;              /* philosopher has finished eating */
      test(LEFT);                       /* see if left neighbor can now eat */
      test(RIGHT);                      /* see if right neighbor can now eat */
      up(&mutex);                       /* exit critical region */
}

void test(i)                            /* i: philosopher number, from 0 to N–1 */
{
      if (state[i] == HUNGRY && state[LEFT] != EATING && state[RIGHT] != EATING) {
            state[i] = EATING;
            up(&s[i]);
      }
}
```

Solution to dining philosophers problem (part 2)

The Readers and Writers Problem

```
typedef int semaphore;                  /* use your imagination */
semaphore mutex = 1;                    /* controls access to 'rc' */
semaphore db = 1;                       /* controls access to the database */
int rc = 0;                             /* # of processes reading or wanting to */

void reader(void)
{
    while (TRUE) {                      /* repeat forever */
        down(&mutex);                   /* get exclusive access to 'rc' */
        rc = rc + 1;                    /* one reader more now */
        if (rc == 1) down(&db);         /* if this is the first reader ... */
        up(&mutex);                     /* release exclusive access to 'rc' */
        read_data_base( );              /* access the data */
        down(&mutex);                   /* get exclusive access to 'rc' */
        rc = rc – 1;                    /* one reader fewer now */
        if (rc == 0) up(&db);           /* if this is the last reader ... */
        up(&mutex);                     /* release exclusive access to 'rc' */
        use_data_read( );               /* noncritical region */
    }
}


void writer(void)
{
    while (TRUE) {                      /* repeat forever */
        think_up_data( );               /* noncritical region */
        down(&db);                      /* get exclusive access */
        write_data_base( );             /* update the data */
        up(&db);                        /* release exclusive access */
    }
}
```

A solution to the readers and writers problem